ORDER FLOW ENGINE

Engineering Product Specification

v1.1 — Updated with Logging System, Part B Integration Fixes, Agent Pipeline

INTERNAL DOCUMENT — FOR ENGINEERING TEAM ONLY

NEW — Logging System Architecture (util/logger.h + logger.cpp)

The engine uses a structured, async, 5-level logging system implemented in util/logger.h (macros + inline interface) and src/util/logger.cpp (background IO thread). This section documents the complete architecture, usage conventions, and known implementation fixes.

Architecture Overview

Component	Implementation	Purpose
LogLevel enum	util/logger.h	5 levels: TRACE(0), DEBUG(1), INFO(2), WARN(3), ERROR(4), OFF(5)
OFE_LOG macro	util/logger.h	Master macro that all LOG_* macros expand to. Checks is_enabled() before building record.
LOG_TRACE / LOG_DEBUG	util/logger.h	Compiled to empty do{}while(0) in Release builds (when NDEBUG defined). Zero cost in production.
LOG_INFO / LOG_WARN / LOG_ERROR	util/logger.h	Always compiled — these represent operational events and failures that must be captured.
Logger singleton	src/util/logger.cpp	Background IO thread drains a std::queue of LogRecords. Writes to file + console.
LoggerImpl (PIMPL)	src/util/logger.cpp	Private implementation: file rotation, queue management, ANSI colour rendering.
LogConfig struct	util/logger.h	Configuration: log_dir, level, max_file_size_mb, max_files, console, color_console, flush_every_write

Log Line Format

```
2026-06-07 14:32:07.291847362 | DEBUG | T:ab12cd34 | bar_engine            | Bar closed: symbol=ES O=4502 H=4508 L=4501 C=4507 vol=12450  [bar_engine.cpp:187]
```

Field	Width	Content
Timestamp	29 chars	YYYY-MM-DD HH:MM:SS.nnnnnnnnn (nanosecond precision)
Level	5 chars	TRACE / DEBUG / INFO / WARN / ERROR
Thread	10 chars	T:XXXXXXXX — lower 32 bits of thread hash (hex)
Module	20 chars	Left-aligned module name (e.g. bar_engine, tick_router, delta_engine)
Message	variable	The formatted log message
Source	optional	[filename.cpp:line] — appended only for DEBUG and TRACE levels

Logging Conventions Per Module

Level	When to Use	Example Modules	Hot Path Safe?
TRACE	Every tick-level event — per-tick routing, per-tick bar accumulation, per-tick delta update	tick_router (every routed tick), bar_engine (bar open), tick_record (excluded ticks)	YES — compiled away in Release (NDEBUG)
DEBUG	Per-bar events — bar close with OHLCV, delta summary, imbalance scan results	bar_engine (bar close), delta_engine (bar delta/CVD), imbalance (scan summary)	NO — always compiled. Keep message construction cheap.
INFO	Lifecycle and signal events — symbol registered, session opened, signal detected, zone created	tick_router (register/unregister), delta_engine (surge detected), imbalance (stacked zone)	NO — infrequent events only
WARN	Recoverable anomalies — ring buffer full (tick dropped), unknown symbol, feed silent, calibration insufficient	tick_router (every 100th drop), tick_router (every 1000th unknown), feed_manager (failover trigger)	NO — should be rare
ERROR	Failures requiring attention — feed disconnect, license invalid, exception caught in run_loop	bar_engine (invalid config), license (fingerprint mismatch), symbol_worker (caught exception)	NO — must always be captured

File Rotation and Retention

- Daily rotation: ofe_2026-06-07.log, ofe_2026-06-08.log, ...
- Size rotation: when file exceeds max_file_size_mb (default 100MB), rotates to .1
- Retention: keeps max_files (default 7) daily logs, purges oldest
- Flush policy: WARN and ERROR are flushed immediately. TRACE/DEBUG/INFO are buffered.
- Console: ANSI colour codes (Grey=TRACE, Cyan=DEBUG, White=INFO, Yellow=WARN, BoldRed=ERROR)

CRITICAL: OFE_LOG Macro Implementation Notes

The OFE_LOG macro has specific implementation constraints discovered during Part B integration. These MUST be preserved in any future modifications:

> ⚠ *Parameter naming: the macro uses 'ofe_lvl' and 'ofe_mod' as parameter names — NOT 'level' or 'module'. Using 'level' or 'module' causes name collision with LogRecord struct fields during macro expansion.*

⚠️ *String literal handling: the module parameter is assigned via std::string(ofe_mod) — NOT via direct assignment _rec.module = (ofe_mod). Direct assignment causes the C preprocessor to substitute the string literal as a field name, resulting in _rec."tick_router" which is invalid C++.*

⚠️ *Format strings: never use {:#x} in LOG_* calls — the # character is interpreted as the preprocessor stringification operator inside variadic macros. Use {:x} instead.*

```
#define OFE_LOG(ofe_lvl, ofe_mod, ...)                      \
    do {                                                     \
        if (::ofe::util::Logger::is_enabled(ofe_lvl)) {     \
            ::ofe::util::LogRecord _ofe_rec;                 \
            _ofe_rec.level  = (ofe_lvl);                     \
            _ofe_rec.module = std::string(ofe_mod);  // <--  \
            _ofe_rec.message = Logger::format(__VA_ARGS__);  \
            Logger::write(std::move(_ofe_rec));              \
        }                                                     \
    } while(0)
```

NEW — Part B Integration Fixes (Build Verification Session)

Part B was a deliberate pause in coding to build, link, and run all Session 1 code. The following bugs were discovered and fixed during this phase. These are documented here to prevent regression.

Fix ID	File	Issue	Root Cause	Fix Applied
PB-01	logger.h	OFE_LOG macro: _rec.level collides with LogLevel::TRACE in expansion	Macro parameter 'level' matched LogRecord field name during expansion	Renamed to 'ofe_lvl'
PB-02	logger.h	OFE_LOG macro: _rec.module expands to _rec."tick_router"	Macro parameter 'module' substituted with the string literal argument	Used std::string(ofe_mod) constructor
PB-03	tick_record.h	make_trade() missing noexcept in declaration, present in definition	Declaration/definition mismatch	Added noexcept to declaration
PB-04	lee_ready.cpp	classify_inplace() left tick.side as default (ASK=0) for quote ticks	Quote processing path did not set side field	Added tick.side = TickSide::UNKNOWN for quote ticks
PB-05	CMakeLists.txt	Empty add_executable() for ofe_server and ofe_tick_gen	Sources commented out but targets still declared	Commented out entire target blocks
PB-06	CMakeLists.txt	Generator expressions $<$<CONFIG:Debug>:-DDEBUG> malformed	Semicolons needed between flags in generator expressions	Simplified to separate add_compile_options calls
PB-07	imbalance_detector.cpp	config_.absorption_vol_threshold not found	Header uses absorption_vol_thresh, impl used _threshold	Changed to absorption_vol_thresh
PB-08	*.cpp	Format strings with {:#x} caused preprocessor errors	# inside variadic macro arg treated as stringification operator	Changed all {:#x} to {:x}
PB-09	tick_router.cpp	Missing <mutex> and <shared_mutex> includes	Used shared_lock/unique_lock without including headers	Added includes

NEW — Agent-Based Development Pipeline

The project uses specialised AI agents for each development phase. Each agent has a defined role, inputs, outputs, and gate criteria. Engineers start any session by reading PROGRESS.md, then using the appropriate agent prompt.

Agent	Phase	Role	Gate
AGT-01	Architecture	Translate spec → architecture doc, layer diagram, data flow	GATE-01
AGT-02	Interface	Generate all C++ headers (.h) — zero implementation	GATE-02 (COMPLETED — 19 headers)
AGT-03	Core Engine	Implement src/core/*.cpp (tick pipeline, bar engine, worker)	GATE-03
AGT-04	Analytics	Implement src/analytics/*.cpp (delta, imbalance, VP, VWAP, signals)	GATE-04
AGT-05	Data Feeds	Implement src/feed/*.cpp (IQFeed TCP, Kinetick bridge, eSignal)	GATE-05
AGT-06	NinjaTrader	Build NT8 C# AddOn — footprint chart renderer, overlays	GATE-06 (14 acceptance criteria)
AGT-07	API	Build Go REST/WebSocket server + Rust OFE-Script evaluator	GATE-07
AGT-09	License	Hardware fingerprint + RSA-2048 engine + billing webhooks	GATE-08
AGT-10	Tests	Generate gtest suites, run with log dump, analyse failures	Runs parallel
AGT-12	Security	Binary protection, Themida/VMProtect, anti-tamper verification	Pre-launch
AGT-13	DevOps	CMake, CI/CD, Docker, AWS deployment, monitoring	Continuous
AGT-14	Review	Final code audit — formula correctness, security, performance	Pre-launch

Agent prompts are stored in agents/prompts/ in the repo. Each prompt contains the full context, rules, and acceptance criteria for that phase. Paste the prompt at the start of a Claude session to activate that agent.

The AGT-10 Test Agent has a MANDATORY log dump protocol: every test run writes structured output to test_logs/ with headers, timestamps, XML results, and failure analysis format.

NEW — Session 1 Implementation Results

Session 1 produced 9 implementation files + 3 test files. All code compiled cleanly after Part B fixes and all 56 unit tests passed.

File	Lines	Key Methods	Logging Added
src/util/logger.cpp	~250	Logger::init/shutdown/write, LoggerImpl::io_loop/rotate/format	N/A (IS the logger)
src/core/tick_record.cpp	~50	make_trade(), should_exclude_tick()	TRACE on each excluded tick
src/core/ring_buffer.cpp	~120	try_push/try_pop/peek/size_approx + explicit instantiations	None (hot path — no logging)
src/core/tick_router.cpp	~130	register_symbol, unregister_symbol, route (const& and &&)	INFO register/unregister, WARN drop/unknown (throttled)
src/core/lee_ready.cpp	~90	classify_trade, update_quote, classify_inplace	None (hot path)
src/core/bar_types.cpp	~80	BarSize::label/is_valid, BarRecord::get_level/range_ticks	None (helpers)
src/core/bar_engine.cpp	~280	BarSeries::on_tick/open_new_bar/ close_current_bar, BarEngine	DEBUG bar close, TRACE bar open, ERROR invalid config
src/analytics/delta_engine.cpp	~240	on_tick, on_bar_close, detect_surge/extreme/divergence/small _range	DEBUG bar delta, INFO surge detection
src/analytics/ imbalance_detector.cpp	~350	detect_all, detect_single, detect_stacked, detect_absorption/trapped	DEBUG scan summary, INFO stacked zone creation

Test Results Summary

Test Suite	Tests	Status
TickRecord	22	ALL PASSED — size invariant 64B, factory, accumulatable, quote, exclude_tick
RingBuffer	12	ALL PASSED — push/pop, overflow, FIFO, concurrent 100K, wrap-around

LeeReadyTest	22	ALL PASSED — 5 rules, IQFeed override, zero-tick, float precision
TOTAL	56	56/56 PASSED

Document Library — Complete Catalog

All specification documents generated during the project:

Document	Version	Audience	Key Content
OrderFlow_Spec_v4_Master	v4.0	All	Master functional spec — 23 sections, all indicators
OFE_Engineering_Product_Spec	v1.1 (this doc)	Engineers	Architecture, performance reqs, error handling, security, tests, logging
OFE_User_Guide	v1.0	Traders	What order flow is, how to read footprints, 4 trade setups, installation
OrderFlow_Execution_Plan	v1.0	PM/Agents	14 agents, 6 phases, 28 weeks, 8 human gates
OrderFlow_Indicator_Formulas	v1.0	Engineers	All 26 indicator formulas (standalone reference)
OrderFlow_Parameter_Calibration_Spec	v1.0	Engineers	AUTO/SEMI-AUTO/MANUAL for all 40 parameters, cold start warm-up
OrderFlow_FootprintChart_Spec	v1.0	AGT-06	Cell colours, Z-order, overlays, NT8 rendering, 14 acceptance criteria
OrderFlow_External_Developer_SDK_Spec	v1.0	API devs	REST endpoints, WebSocket topics, OFE-Script, backtesting
OrderFlow_DataFeed_MultiSymbol_Spec	v1.0	AGT-05	IQFeed/Kinetick/eSignal deep-dive + 1-to-1000 scaling
OrderFlow_MultiSymbol_Scaling_Spec	v1.0	AGT-03	Symbol-sharded architecture, concurrent symbol processing
OrderFlow_Timestamp_Specification	v1.0	All	Signal vs snapshot timestamps, 4-timestamp model

END — EPS v1.1 | Updated with: Logging System (§NEW), Part B Fixes (§NEW), Agent Pipeline (§NEW), Session 1 Results (§NEW), Document Library (§NEW)